

Java Advanced Schulung 2026

Lea Föcke & Phil Steinhorst



Wer sind wir?



Dr. Lea Föcke – IT Komposit Vertrag Zentral
seit Februar 2023 in der Provinzial
Themen: Spring; REST-APIs; Architektur



Phil Steinhorst – AE Leben 1
seit Juni 2024 in der Provinzial
Themen: Java-Backends, Spring Batch, DevOps @ Neues Haus

Wer seid ihr? Ein kurzes Kennenlernen

Sucht euch zu eurem Vornamen ein Adjektiv aus, das **mit dem gleichen Anfangsbuchstaben** beginnt!

Grundsätzlicher Ablauf

- Dienstags, mittwochs und donnerstags
- 09:00 Uhr bis 16:00 Uhr mit einstündiger Mittagspause
- Kürzere Pausen zwischendurch
- Kleinere Input-Phasen am Vormittag, gefolgt von aktiven Übungsphasen
- Eigenständiges Arbeiten am Nachmittag

Plan für heute

09:00 Uhr: Kurzes Kennenlernen ✓

09:15 Uhr: Wiederholung von Grundbegriffen

10:00 Uhr: Generics

12:00 Uhr: Mittagspause

13:00 Uhr: Datenstrukturen und Collections

16:00 Uhr: Ende

Wiederholung (?)

Diskutiert mit euren Nachbar*innen, was ihr unter folgenden Begriffen versteht:

Objekt	Klasse	Methode
Rückgabotyp	Methodensignatur	Konstruktor
Attribut	Lokale Variable	Scope
Datentyp	Array	<code>null</code>
Sichtbarkeit	Exception	Vererbung

Aufgabe 1

Siehe `Tag1_1_WasDas.pdf` .

Dokumentation von Programmcode

- Sprechende, sinnvolle Bezeichner unterstützen die Nachvollziehbarkeit...
- ... sind aber kein Ersatz für eine Code-Dokumentation!
- Möglichkeiten, Java-Code zu dokumentieren:
 - Inline-Kommentare mit `//`
 - Block-Kommentare mit `/* ... */`
 - **Javadoc**-Blöcke mit `/** .. */` zur Dokumentation von Programmteilen

Dokumentation mit Javadoc

- Mehrzeilige Blöcke mit `/** .. */` zur Dokumentation u.a. von
 - Programmbestandteilen wie Klassen, Interfaces, Enums, ...
 - Methoden
 - Attributen
- Anreicherung durch **Tags** wie `@author`, `@param`, `@return`, `@link` ...
- Kann im HTML-Format ausgegeben werden.
- **Aufgabe:** Ergänzen der Klasse `MeineListe` um Javadoc-Dokumentation und Code-Kommentare.
- Übersicht: <https://www.baeldung.com/javadoc>

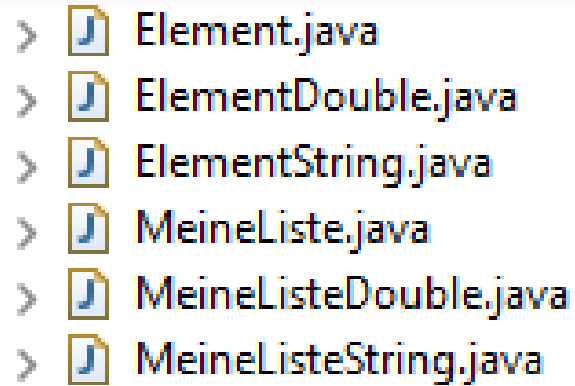
Kurze Pause!

Rückblick: MeineListe

- Die Klasse `MeineListe` speichert beliebig viele `int` -Werte.
- **Frage:** Was kann man tun, wenn man zusätzlich mit einer Liste von `String` -Werten arbeiten möchte?

Naive Möglichkeit 1

- Separate Klassen für jeden Datentyp erstellen.



```
> Element.java
> ElementDouble.java
> ElementString.java
> MeineListe.java
> MeineListeDouble.java
> MeineListeString.java
```

- Führt zu viel redundantem Code.
 - Fehleranfällig.
 - Mühsam zu warten.

Naive Möglichkeit 2

- "Allgemeine" Liste für `Object` erstellen und Typecasts nutzen.
- Nicht typsicher! Gültigkeit von Typecasts sind erst zur Laufzeit überprüfbar.

```
public class ElementObject {  
    private Object wert;  
    private ElementObject next;  
    /* ... */  
}  
  
MeineListeObject ls = new MeineListeObject();  
ls.anhaengen(Integer.valueOf(42)); // geht, denn jeder Integer ist auch Object  
ls.anhaengen("25");               // geht, denn jeder String ist auch Object  
  
int x = (Integer) ls.gibWertAnPosition(0); // geht, denn 42 ist Integer  
int y = (Integer) ls.gibWertAnPosition(1); // Fehler, aber erst zur Laufzeit (!)
```

Generics

- **Lösung:** Erzeugen einer generischen Klasse durch Verwendung eines Typparameters:

```
public class Element<T> {  
    private T wert;  
    private Element<T> next;  
    /* ... */  
}
```

- Typparameter `T` ist Platzhalter für Datentypen, die erst beim Instanziiieren eingesetzt werden:

```
Element<Integer> x = new Element<Integer>(42);  
Element<Integer> x = new Element<>(42); // gleichwertig  
var x = new Element<>(42); // seit Java 10 möglich, aber nicht immer klar
```

Generics: Blick unter die Haube

- Generics werden in Java durch **Typlöschung** realisiert.
- Bedeutet: Die Information, welcher konkrete Typ für einen Parameter eingesetzt wird, ist nur zur Compilezeit bekannt.
- Der Compiler macht aus allen Typparametern `Object` und ergänzt Typecasts – wie bei der naiven Möglichkeit 2.
- Daher sind keine Konstruktoraufrufe für einen Typparameter `T` möglich:

```
public class Element<T> {  
    private T wert;  
  
    /* in irgendeiner Methode: */  
    wert = new T(); // funktioniert nicht  
}
```

Generics: Typparameter

- Typparameter sind üblicherweise einzelne Großbuchstaben
 - Wörter ebenfalls möglich, aber Verwechslungsgefahr mit Klassen
- Mehrere Typparameter gleichzeitig möglich:

```
public class Paar<S, T> {  
    private S erstes;  
    private T zweites;  
    /* ... */  
}
```

```
Paar<Integer, String> x = new Paar<>(42, "Hallo");
```

- Für jedes Vorkommen des Typparameters in der generischen Klasse wird beim Instanziiieren immer derselbe Typ eingesetzt:

```
public class Element<T> {  
    private T wert;  
    private Element<T> next;  
    /* ... */  
}
```

```
Element<String> x = new Element<>("Hallo!");  
Element<Integer> y = new Element<>(42);  
x.setNext(y); // geht nicht, da x.next anderer Typ als y!
```

- In `x` ist `wert` vom Typ `String` und `next` vom Typ `Element<String>`.
- Anders formuliert: `x` und `x.next` sind **immer** vom selben Typ.
- Und wenn `next` stattdessen auf ein **beliebiges** `Element` verweisen können soll?

Wildcard-Operator

- Wenn der konkrete Typ des Parameters keine Rolle spielt, kann man den Wildcard-Operator `?` nutzen:

```
public class Element<T> {  
    private T wert;  
    private Element<?> next;  
    /* ... */  
}  
  
Element<String> x = new Element<>("Hallo!");  
Element<Integer> y = new Element<>(42);  
x.setNext(y); // geht jetzt!
```

- **Frage:** Welchen Typ hat `x.getNext().getWert()` ?

Wildcards und Typlöschung

```
public class Element<T> {  
    private Element<?> next;  
    private T wert;  
}  
  
Element<Integer> x = new Element<>(27);  
Element<Integer> y = new Element<>(42);  
x.setNext(y);  
  
y.getWert() + y.getWert(); // funktioniert  
x.getNext().getWert() + x.getNext().getWert(); // funktioniert nicht
```

- Die Typlöschung macht aus Wildcards den allgemeinsten Datentyp `Object`.
- Aus "Sicht" von `x` ist daher unbekannt, dass `x.getNext()` ein `Element<Integer>` ist – und `x.getNext().getWert()` ein `Integer`.

Auswirkungen der Typlöschung

- So etwas funktioniert nicht:

```
public class Element<T> {  
    private T wert;  
  
    public double getKommazahl() {  
        return wert.doubleValue(); // geht nicht  
    }  
}
```

- Erinnerung: Die Typlöschung macht aus Typparametern `Object`.
- Für Objekte eines parametrisierten Typs steht daher (zunächst) nur die Funktionalität von `Object` zur Verfügung – und `Object` kennt keine Methode `doubleValue()`.

Schranken für Typparameter

- Typparameter lassen sich durch das Schlüsselwort `extends` **nach oben beschränken**:

```
public class Element<T extends Number> {  
    private T wert;  
  
    public double getKommazahl() {  
        return wert.doubleValue(); // geht jetzt!  
    }  
}
```

- `<T extends Number>` bedeutet: Für `T` muss `Number` oder eine **Unterklasse** von `Number` eingesetzt werden.
- Für `T` steht daher die Funktionalität von `Number` zur Verfügung.

Mehr zu Generics

- Grundlagen zum Nachlesen: <https://www.baeldung.com/java-generics>
- Generische Methoden – auch in nicht-generischen Klassen:
https://www.tutorialspoint.com/java_generics/java_generics_methods.htm
- Typschränken für Wildcards -- auch in die andere Richtung:
https://www.tutorialspoint.com/java_generics/java_generics_upper_bounded.htm

Mittagspause!

Weiter geht's um 13:00 Uhr!

Offene Fragen?



Wiederholung: Interfaces

- **Interfaces** sind abstrakte Datentypen, die typischerweise nur Methodensignaturen mit Rückgabetypen enthalten.
- *abstrakt* bedeutet: Kann nicht instanziiert werden.
- Ein Interface gibt die **Schnittstelle** vor, d.h., welche Funktionalitäten eine Klasse bereitstellen muss, die dieses Interface implementiert.
- Mit anderen Worten: Interfaces definieren, **was** eine Klasse leistet, aber nicht **wie**.

```
public interface Tier {  
    String gibLaut();  
    void schlafe(int dauer);  
}
```

- Damit wirklich klar ist, was die Anforderungen an die implementierenden Klassen sind, ist eine saubere Dokumentation unerlässlich!

```
public interface Tier {  
    /**  
     * Gibt einen String zurück, der den Tierlaut enthält.  
     * @return String mit Tierlaut.  
     */  
    String gibLaut();  
  
    /**  
     * Lässt das Tier für eine gewisse Zeit schlafen.  
     * @param dauer Schlafzeit in Stunden.  
     */  
    void schlafe(int dauer);  
}
```

- Eine Klasse, die das Interface implementiert, muss **mindestens** die Methoden des Interface umsetzen.

```
public class Hund implements Tier {  
    @Override  
    public String gibLaut() { return "Wuff!"; }  
  
    @Override  
    public void schlafe(int dauer) {  
        for (int i = 0; i < dauer; i++)  
            System.out.println("...zzz...");  
        System.out.println(gibLaut());  
    }  
}
```

- `@Override` überprüft, dass man *wirklich* eine Methode des Interface implementiert.

Mini-Aufgabe: Interface implementieren

Implementiert das Interface `Tier` für eine weitere konkrete Tierart und stellt die von euch implementierte Klasse in den Chat!

```
public class Hund implements Tier {  
    @Override  
    public String gibLaut() { return "Wuff!"; }  
  
    @Override  
    public void schlafe(int dauer) {  
        for (int i = 0; i < dauer; i++)  
            System.out.println("...zzz...");  
        System.out.println(gibLaut());  
    }  
}
```

Verwendung von Interfaces

- Interfaces können grundsätzlich so verwendet werden wie jeder andere Referenzdatentyp auch.

```
Tier[] a = new Tier[3];

// a[0] = new Tier();    // geht nicht, da Tier nicht instanziiierbar!
a[0] = new Hund();
a[1] = new Katze();
a[2] = new Maus();

for (int i = 0; i < a.length; i++) {
    Tier t = a[i];
    t.schlafe(3);    // Welche Methode wird hier verwendet?
}
```

- Insbesondere kann man von Interfaces erben und sie für Typparameter-Schranken verwenden:

```
public interface Haustier extends Tier {...}  
  
public class Element<T extends Tier> { ... }
```

- Was Interfaces sonst noch enthalten können:
 - Konstanten
 - Statische Methoden
 - Default-Methoden
- Einen Überblick dazu: <https://www.baeldung.com/java-interfaces>

Wiederholung: Arrays

- Arrays (dt.: *Felder*) bündeln mehrere Entitäten eines Datentyps zu einem Objekt.
- Erzeugung des Arrays durch Angabe der Anzahl der Elemente...

```
int[] a = new int[6]; // erzeugt Array aus 6 int; sind zunächst alle 0  
String[] b = new String[4]; // erzeugt Array aus 4 String; sind zunächst alle null
```

- oder Definieren der einzelnen Elemente:

```
int[] c = { 4, 8, 15, 16, 23, 42 };  
String[] d = {"Hallo", "wie", "geht's", "dir?"};
```

```
// direktes Adressieren einzelner Werte des Arrays  
a[0] = 17;  
a[1] = 38;  
a[2] = a[0] + a[1];
```

- Beim Erzeugen eines Arrays wird ein **zusammenhängender Speicherbereich** reserviert, in welchem alle Elemente des Arrays hintereinander abgelegt werden.
- Technisch speichert die Variable `a`, an welcher Speicheradresse das erste Element steht.
- Alle Elemente eines Datentyps sind gleich groß (unabhängig vom Wert!).
- Dadurch lassen sich anhand der Indizes die Elemente **direkt adressieren**.

Beispiel

```
int[] c = { 4, 8, 15, 16, 23, 42 };  
System.out.println(c[2]);
```

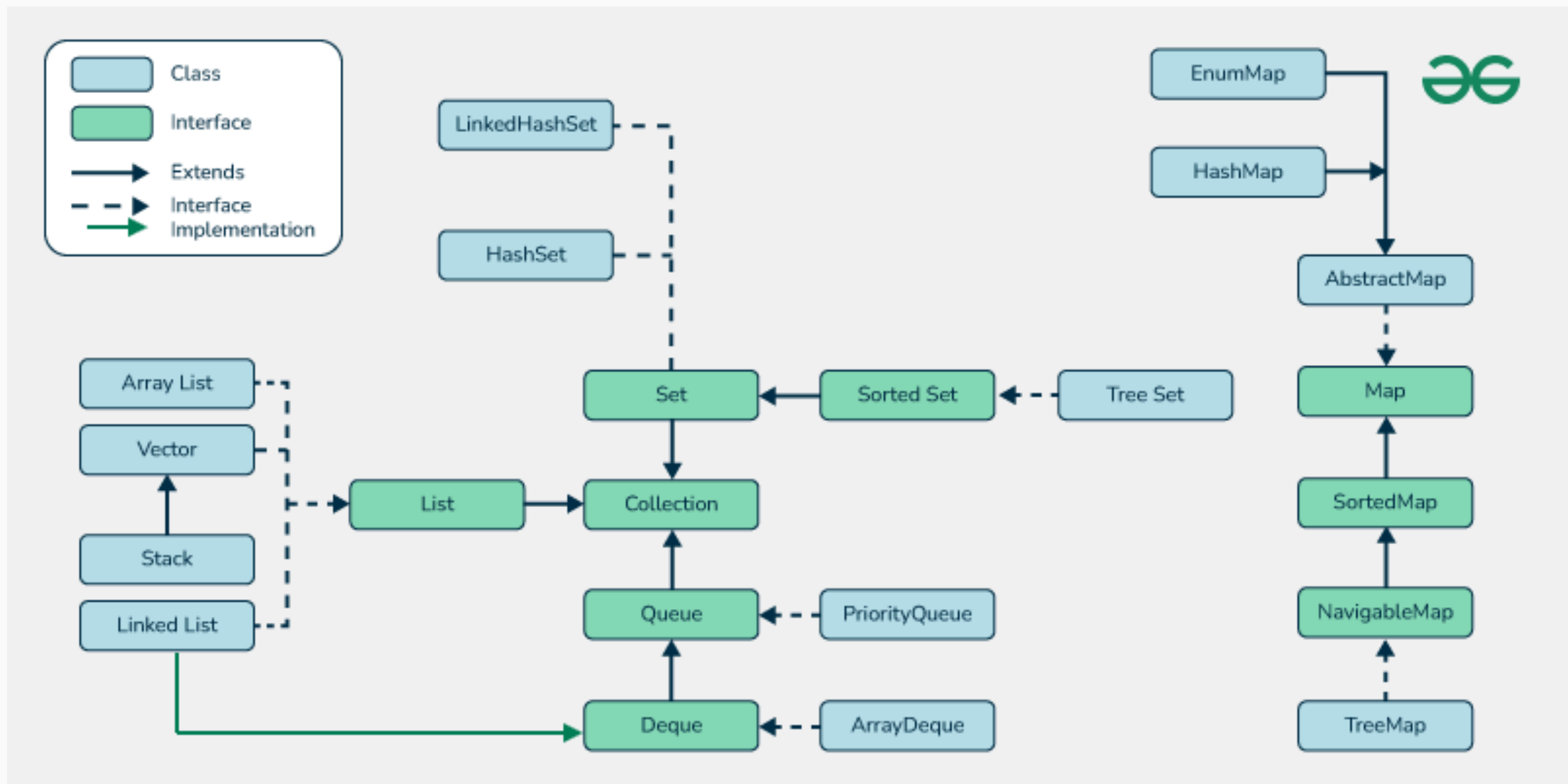
- Jeder `int`-Wert ist 4 Byte groß (unabhängig vom konkreten Wert)
 - Es werden $4 \cdot 6 = 24$ Byte Speicher für das Array reserviert.
- Variable `c` enthält Startadresse des Arrays (z.B. Speicher-Byte 1432).
- Beim Zugriff auf `c[2]`: Rechne Startadresse aus `c` + $2 \cdot 4$ Byte.
- → Das Programm greift auf Speicheradresse 1440 zu und findet dort den Wert 15.

Vor- und Nachteile

- Dadurch, dass alle Elemente hintereinander stehen und direkt adressiert werden können, sind Arrays sehr effizient...
- ... aber leider auch sehr unflexibel:
 - Keine Möglichkeit, Arrays nachträglich zu vergrößern oder zu verkleinern.
 - Keine Möglichkeit, Elemente einzufügen oder zu entfernen.
 - `ArrayIndexOutOfBoundsException` -Gefahr
- Die Umgehung dieser Nachteile ist zwar möglich, erfordert aber zusätzlichen Programmieraufwand.

Dynamische Datenstrukturen

- Die Umgehung dieser Nachteile ist zwar möglich, erfordert aber zusätzlichen Programmieraufwand.
- Alternative: **Dynamische Datenstrukturen**, die zur Laufzeit wachsen und schrumpfen können.



<https://www.geeksforgeeks.org/collections-in-java-2/>

Kategorisierung von Datenstrukturen

Verschiedene Datenstrukturen können nach ihren Eigenschaften kategorisiert werden:

- **Iterierbarkeit:** Elemente können der Reihe nach durchlaufen werden.
- **Ordnung:** Elemente haben (nach Einfügen) eine feste Reihenfolge.
- **Wahlfreier Zugriff:** Zugriff auf beliebiges Element möglich.
- **Sortierung:** Elemente werden nach einem bestimmten Kriterium sortiert.
- **Duplikate:** Gleiche Elemente können mehrfach enthalten sein.
- **Effizienz:** Wie effizient kann welche Operation ausgeführt werden?
- ...

Iterierbar – ge**O**rdnet – **W**ahlfreier Zugriff – **S**ortiert – **D**uplikate

	I	O	W	S	D
ArrayList	✓	✓	✓	—	✓
LinkedList	✓	✓	—	—	✓
Stack	✓	✓	—	—	✓
Vector	✓	✓	✓	—	✓
ArrayDeque	✓	✓	—	—	✓
PriorityQueue	✓	✓	—	✓	✓

Iterierbar – ge**O**rdnet – **W**ahlfreier Zugriff – **S**ortiert – **D**uplikate

	I	O	W	S	D
HashSet	✓	—	—	—	—
LinkedHashSet	✓	✓	—	—	—
TreeSet	✓	✓	—	✓	—
HashMap	—*	—	—	—	—**
LinkedHashMap	—*	✓	—	—	—**
TreeMap	—*	✓	—	✓	—**

*: Iteration über die Menge der Keys möglich.

** : Jeder Key darf höchstens einmal enthalten sein (Values grundsätzlich mehrfach).

Weiterführende Links

- <https://www.baeldung.com/java-choose-list-set-queue-map>